



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

**Wydział Informatyki**  
**Środowiska Udostępniania Usług**  
Grupa 4 - czwartek 13:15

***MCP-PK***

*Obserwowalność Kubernetes z Prometheusem kontrolowanym  
przez serwer MCP*

*Kubernetes Observability with Prometheus controlled by MCP  
server*

Autorzy: Dominik Drózdź, Szymon Miękina, Magdalena Pabisz, Marcin Walendzik

Kraków, 2026

# Spis treści

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| <b>1</b>  | <b>Wprowadzenie</b>                               | <b>3</b>  |
| <b>2</b>  | <b>Podstawy teoretyczne i stos technologiczny</b> | <b>4</b>  |
| 2.1       | Kubernetes . . . . .                              | 4         |
| 2.2       | Prometheus . . . . .                              | 4         |
| 2.3       | MCP Server . . . . .                              | 5         |
| 2.4       | Large Language Model . . . . .                    | 5         |
| 2.5       | Grafana . . . . .                                 | 5         |
| <b>3</b>  | <b>Opis studium przypadku</b>                     | <b>6</b>  |
| 3.1       | Aplikacja testowa: Microservices-demo . . . . .   | 6         |
| 3.1.1     | Architektura i zasada działania . . . . .         | 6         |
| 3.1.2     | Uzasadnienie wyboru . . . . .                     | 7         |
| 3.2       | Opis przypadku użycia . . . . .                   | 7         |
| <b>4</b>  | <b>Wysokopoziomowy opis architektury</b>          | <b>8</b>  |
| <b>5</b>  | <b>Szczegółowy opis architektury</b>              | <b>9</b>  |
| <b>6</b>  | <b>Konfiguracja środowiska</b>                    | <b>10</b> |
| <b>7</b>  | <b>Sposób instalacji</b>                          | <b>11</b> |
| <b>8</b>  | <b>Wdrożenie wersji demonstracyjnej</b>           | <b>12</b> |
| <b>9</b>  | <b>Opis demonstracji</b>                          | <b>13</b> |
| <b>10</b> | <b>Podsumowanie i wnioski</b>                     | <b>14</b> |
|           | <b>Spis rysunków</b>                              | <b>15</b> |
|           | <b>Spis tabel</b>                                 | <b>16</b> |

# Rozdział 1

## Wprowadzenie

Obserwowalność (observability) jest kluczowym elementem nowoczesnych systemów cloud-native. Umożliwia pomiar, analizę oraz zrozumienie wewnętrznego stanu systemu na podstawie jego zewnętrznych wyników. Opiera się na trzech głównych typach danych: metrykach, które pozwalają monitorować ogólną efektywność systemu, logach, które opisują zdarzenia zachodzące w systemie, oraz śladach, które umożliwiają śledzenie przepływu operacji.

Współczesne aplikacje coraz częściej składają się z niezależnych mikroserwisów i działają w środowiskach kontenerowych, takich jak Kubernetes. W systemach rozproszonych niemożliwe jest ręczne śledzenie zależności pomiędzy usługami i przewidywanie potencjalnych awarii. Z tego względu niezbędne jest zastosowanie narzędzi wspierających obserwowalność. Kubernetes nie posiada wbudowanych mechanizmów monitorowania, dlatego integruje się go z zewnętrznymi narzędziami, takimi jak Prometheus. Dzięki obserwowalności można zrozumieć zależności pomiędzy poszczególnymi usługami, zlokalizować nieefektywne fragmenty kodu czy wolne zapytania do bazy danych. W konsekwencji odpowiednie monitorowanie pozwala zapewnić niezawodny, wydajny i skalowalny system.

Celem projektu jest przygotowanie rozwiązania do obserwowalności aplikacji uruchomionej w środowisku Kubernetes z wykorzystaniem Prometheusa. Dodatkowo infrastruktura odpowiedzialna za obserwowalność jest rozszerzona o serwer MCP (Model Context Protocol), który pozwala na wykorzystanie dużego modelu językowego. Dzięki temu możliwe jest zarządzanie konfiguracją Prometheusa za pomocą poleceń w języku naturalnym. Serwer MCP pełni rolę warstwy pośredniczącej, która przekształca język naturalny na operacje konfiguracyjne wykonywane w Prometheusie. Takie podejście pozwala częściowo zautomatyzować zarządzanie infrastrukturą monitorującą oraz zwiększyć elastyczność i wygodę jej obsługi.

## Rozdział 2

# Podstawy teoretyczne i stos technologiczny

Wybór technologii do tworzenia projektu został częściowo podyktowany sugestiami zawartymi w dokumentach z zajęć laboratoryjnych przedmiotu Środowiska Udostępniania Usług, jak i również powszechnością konkretnych rozwiązań w przypadku danej funkcjonalności. Większość z poniżej opisanych narzędzi jest rozwijana jako technologie typu open-source, co zapewnia często większą swobodę przy korzystaniu z produktu.

### 2.1. Kubernetes

Kubernetes jest narzędziem stworzonym do orkiestracji kontenerów. Stanowi obecnie najpowszechniejszą technologię z tego zakresu. Jej produkcja została rozpoczęta przez Google, a obecnie jest rozwijana jako produkt otwartoźródłowy. Kubernetes opiera swoje działanie o kontrolę nad aplikacjami zamkniętymi w kontenerach. Technologia ta pozwala na dynamiczne zarządzanie usługami zapewniając łatwą skalowalność, kluczową obecnie w wielkich środowiskach usługowych. System dostosowuje automatycznie ilość potrzebnych instancji kontenerów w oparciu o zdefiniowane przez deweloperów reguły oraz obciążenie. W kontekście naszego projektu, warto wspomnieć również, że Kubernetes wystawia endpointy, które służą warstwie observability do czytania danych metrycznych pomagających określić stan serwisów oraz zasobów sprzętowych.

### 2.2. Prometheus

Prometheus to technologia open-source służąca do monitorowania stanów zasobów w złożonych aplikacjach i systemach o wielu usługach. Zarządza on danymi wystawionymi przez aplikację, agregując je w bazie danych typu time-series database. W stosunku do aplikacji działa jako klient, który aktywnie pobiera wyznaczone dane dostępne na endpointach HTTP. Skupia się on na danych numerycznych z zakresu m.in. zużycia zasobów pamięci operacyjnej oraz masowej, ilości wystąpień wyjątków, czasu odpowiedzi aplikacji na zapytania. Dane te są dostępne następnie za pomocą wbudowanego języka zapytań PromQL. Narzędzie to udostępnia również mechanizm powiadamiania wskazanych użytkowników nt. awarii, bądź nieefektywnego zużycia poszczególnych zasobów. W kontekście wielousługowych aplikacji Prometheus jest obecnie pomocnym i skutecznym narzędziem pozwalającym na ciągłe monitorowanie stanu aplikacji, dostosowywanie zasobów pod wskazany cel, będąc jednocześnie dostępnym w formie wolnego oprogramowania.

## 2.3. MCP Server

MCP Server jest warstwą stanowiącą pomost między instrukcją w języku naturalnym a konfiguracją w systemie. Działa w oparciu o otwartoźródłowy protokół internetowy MCP (*Model Context Protocol*). Udostępnia on warstwę komunikacyjną dla modelu, dzięki której może on dynamicznie korzystać z zasobów i funkcji serwera.

## 2.4. Large Language Model

Duży model językowy jest elementem najistotniejszym z perspektywy instrukcji wejściowych. Od strony technicznej jest to specjalnie wytrenowana sieć neuronowa zdolna do generowania tekstu. W przypadku opisywanego projektu powinna stanowić pośrednika, który będzie umiał przetłumaczyć instrukcje użytkownika w języku naturalnym na odpowiednie zapytania techniczne i operacje konfiguracyjne. Jego wyjście kierowane do serwera MCP sprawi, że użytkownik będzie mógł aktywnie konfigurować element observability z pozycji instrukcji sformułowanych "po ludzku".

## 2.5. Grafana

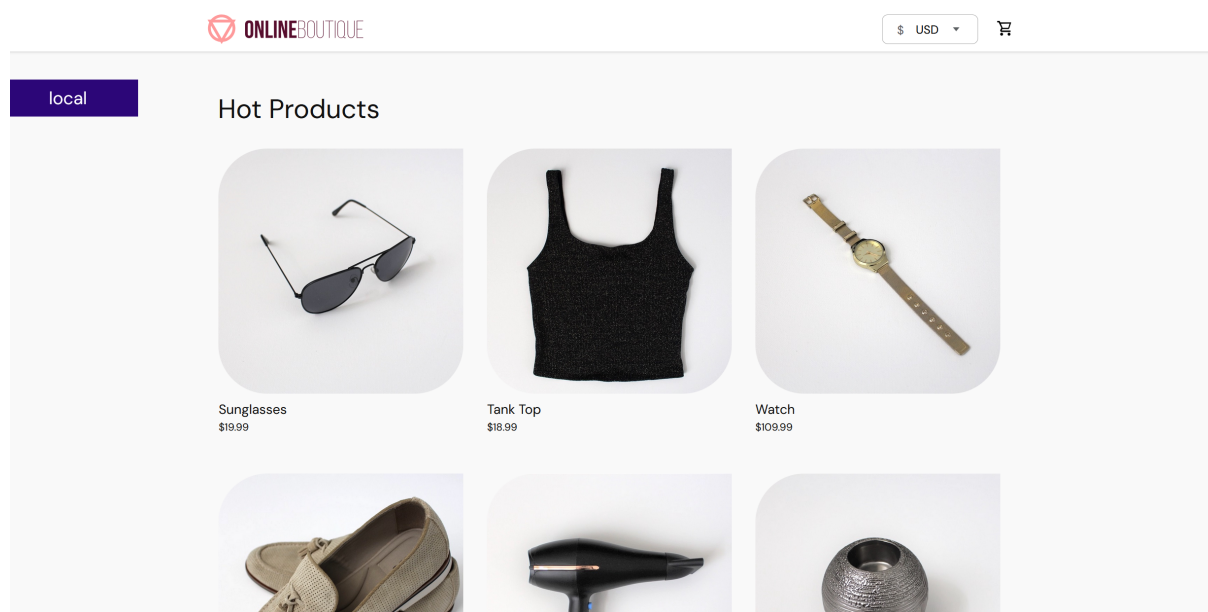
Grafana jest elementem służącym do wizualizacji danych przechowywanych w bazie danych Prometheusa. Jej przewagą jest większa możliwość w kwestii wizualizacji. Dzięki niej można utrzymywać kompozycje różnych wykresów zależnie od celu. Grafana, podobnie jak poprzednie technologie, jest również dostępna jako technologia open-source.

## Rozdział 3

# Opis studium przypadku

### 3.1. Aplikacja testowa: Microservices-demo

W projekcie jako obiekt monitorowania wykorzystano aplikację **Google Microservices Demo** (znaną również pod nazwą *Online Boutique*). Jest to otwartoźródłowa aplikacja demonstracyjna, która symuluje działanie sklepu internetowego. Składa się ona z 11 mikroservisów napisanych w różnych językach programowania (m.in. Go, Python, Java, C#, Node.js), które komunikują się ze sobą za pomocą protokołu gRPC.



Rysunek 3.1: Aplikacja Online Boutique

#### 3.1.1. Architektura i zasada działania

Aplikacja została zaprojektowana w sposób natywny dla chmury (*cloud-native*), co oznacza, że każdy jej komponent pełni ściśle określoną rolę:

- **Frontend:** Serwer WWW napisany w Go, który serwuje interfejs użytkownika.
- **Product Catalog Service:** Dostarcza informacje o produktach z plików JSON.

- **Cart Service:** Zarządza koszykami użytkowników, wykorzystując bazę danych Redis do przechowywania danych tymczasowych.
- **Currency Service:** Odpowiada za przeliczanie walut.
- **Payment Service:** Obsługuje procesy płatności (symulacja).
- **Shipping Service:** Oblicza koszty dostawy.
- **Ad Service:** Serwuje reklamy kontekstowe.

Każdy z serwisów jest odizolowanym kontenerem, co pozwala na niezależne skalowanie komponentów wewnątrz klastra Kubernetes.

### 3.1.2. Uzasadnienie wyboru

Wybór tej konkretnej aplikacji do projektu obserwowalności z wykorzystaniem serwera MCP i Prometheusa jest podyktowany kilkoma kluczowymi czynnikami:

1. **Różnorodność technologiczna:** Dzięki zastosowaniu wielu języków i frameworków, aplikacja stanowi idealne środowisko do testowania uniwersalności narzędzi monitorujących.
2. **Złożoność ruchu sieciowego:** Komunikacja gRPC pomiędzy wieloma ogniwami systemu pozwala na generowanie realistycznych metryk dotyczących opóźnień (latency), liczby zapytań oraz błędów, co jest kluczowe dla demonstracji możliwości Prometheusa.
3. **Gotowość do wdrożenia na Kubernetes:** Aplikacja posiada natywne manifesty YAML oraz wsparcie dla Helm, co znacząco ułatwia jej szybką orkiestrację i integrację z infrastrukturą monitorującą.
4. **Realizm przypadku użycia:** Symulacja sklepu internetowego pozwala na łatwe definiowanie reguł biznesowych i alertów (np. powiadomienie o błędzie w procesie płatności lub zbyt wolnym ładowaniu katalogu produktów), które mogą być następnie modyfikowane za pomocą poleceń w języku naturalnym przez serwer MCP.

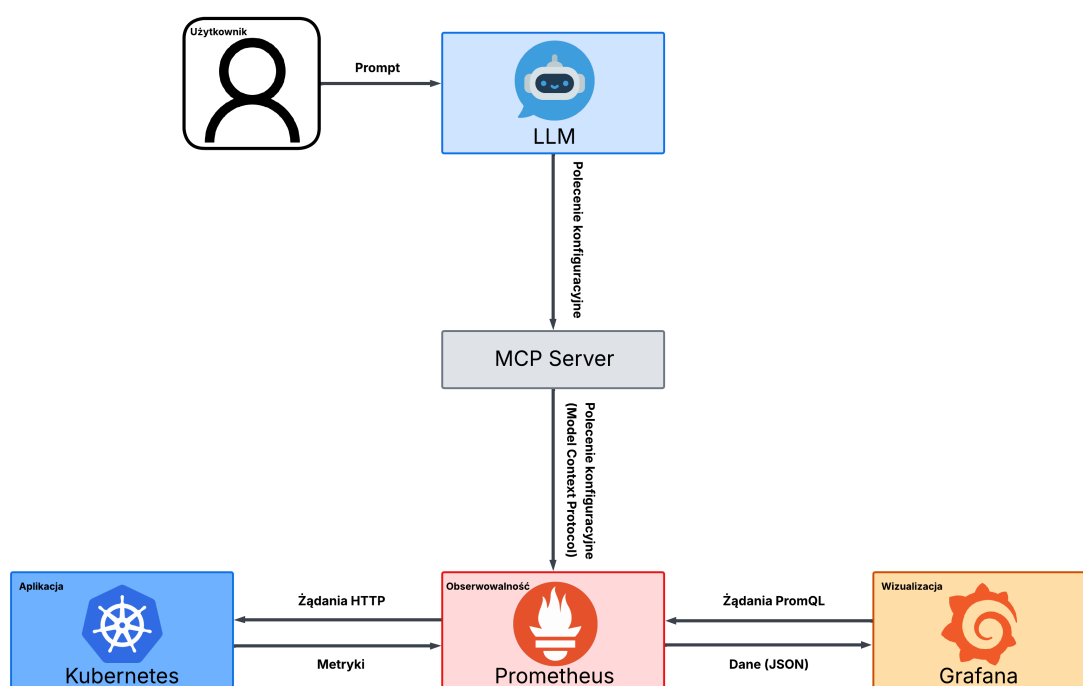
## 3.2. Opis przypadku użycia

Poniżej opisano kolejne operacje, które zostaną przeprowadzone w celu demonstracji przypadku użycia.

1. Uruchomienie klastra **Kubernetesa** z wybraną aplikacją oraz wdrożenie narzędzia monitorującego **Prometheus** wraz z podstawową konfiguracją zbierającą dane dotyczące zużycia CPU, a także integracja **serwera MCP** oraz systemu wizualizacji **Grafana** z Prometheusem.
2. Wydanie poleceń w języku naturalnym przy użyciu dużego modelu językowego, w wyniku których dokonane zostaną następujące zmiany w konfiguracji Prometheusa:
  - zmniejszenie interwału zbierania metryk,
  - dodanie reguły alertowania o wysokim zużyciu procesora,
  - rozszerzenie monitorowania o metrykę zużycia pamięci (memory usage).
3. Zaobserwowanie wprowadzonych zmian w konfiguracji w Grafanie.

# Rozdział 4

## Wysokopoziomowy opis architektury



Rysunek 4.1: Architektura wysokopoziomowa



## **Rozdział 5**

### **Szczegółowy opis architektury**

## **Rozdział 6**

### **Konfiguracja środowiska**

## **Rozdział 7**

### **Sposób instalacji**

## **Rozdział 8**

### **Wdrożenie wersji demonstracyjnej**

## **Rozdział 9**

### **Opis demonstracji**

## **Rozdział 10**

### **Podsumowanie i wnioski**

# Spis rysunków

|     |                                        |   |
|-----|----------------------------------------|---|
| 3.1 | Aplikacja Online Boutique . . . . .    | 6 |
| 4.1 | Architektura wysokopoziomowa . . . . . | 8 |

## **Spis tabel**